LANGCHAIN 1.4+ • ADVANCED SCHEMA EXTRACTION

Structured Outputs & Schema Extraction

Enforcing Type Safety, Validation, and Deterministic Schemas via Pydantic, TypedDict, DataClasses, and Agent response_format

An exhaustive architectural guide to deterministic LLM extraction. Explores how LangChain compiles Python type schemas into function-calling constraints, extracts flat and nested Pydantic models, leverages lightweight TypedDict structures, inspects raw reasoning metadata, and binds structured response formats directly into autonomous agents.

Target Notebook: `5-structure.ipynb`

Generation Date: November 01, 2026

Language & Runtime: Python 3.14 • Virtualenv (.venv)

Core Framework: LangChain 1.4.7 • Pydantic v2 • Groq

B. Learning Objectives

After studying this notebook and study guide, you will master:

- **The Deterministic Schema Guarantee:** Understand why enterprise production applications demand validated data schemas over unstructured generative text.
- **Pydantic BaseModel Compilation:** Construct robust Pydantic schemas with `Field(description=...)` and enforce data types across primitives and nested models.
- **Dual Payload Extraction with include_raw:** Capture raw generation metadata (reasoning traces, token usage, tool call envelopes) alongside validated domain objects.
- **Hierarchical & Nested Schemas:** Design complex multi-level data models containing lists of sub-models and nullable optional attributes (`budget: float | None`).
- **Lightweight Schemas with TypedDict:** Leverage native Python typing (`TypedDict` and `Annotated`) when third-party Pydantic classes are undesirable.
- **Agentic Extraction via response_format:** Integrate structured extraction directly into LangChain's `create_agent` for autonomous extraction loops.

C. Prerequisites & Setup

> **Environment Specification**
> This notebook runs against the high-throughput **Groq LPU Inference Cloud** executing the `openai/gpt-oss-20b` open-weights reasoning model.

Component	Configuration / Package	Purpose
Python Runtime	`Python >= 3.14`	Modern runtime supporting advanced type hinting (`float | None`) and fast native execution.
Virtual Environment	`.venv` (managed via `uv`)	Isolates dependencies defined in `pyproject.toml` .
Validation Libraries	`pydantic>=2.0` , `typing_extensions`	Provides runtime data validation, serialization, and type introspection.
Core Dependencies	`langchain>=1.4.7` , `langchain-groq` , `python-dotenv`	Provides model initialization and `with_structured_output` compiler layer.

> **Security Best Practice**
> Never commit real API keys to version control. Always verify that `.env` is tracked in `.gitignore` and use `.env.example` for team onboarding.

D. Topic Overview: The Shift from Generative Text to Deterministic Schemas

Standard LLM completions return free-form text: strings of paragraphs, markdown bullet points, or informal prose. While pleasant for human reading, software applications, microservices, and databases cannot reliably ingest free-form text without fragile, regex-heavy post-processing.

LangChain solves this through the universal `with_structured_output(schema)` abstraction. Rather than asking the model to "please output JSON" via system prompt formatting instructions, LangChain leverages modern model **tool / function calling mechanisms**:

1. **Schema Compilation:** LangChain inspects the Python schema (Pydantic model or TypedDict) and compiles it into an exact JSON Schema definition.
2. **Tool Binding & Enforcement:** The JSON Schema is transmitted to the provider API as a mandatory tool call. The provider forces the model to emit arguments matching the schema.

3. **Validation & Coercion:** Upon receiving the JSON payload, LangChain feeds it into Pydantic, validating types, enforcing constraints, and returning an instantiated Python object directly to your code.

E. Detailed Notebook Walkthrough & Cell Dissection

Cell 0: Markdown Section Header

```
### Structure
```

Analysis: Establishes the study module exploring structured data extraction.

Cell 1: Model Initialization & Provider Profile Inspection

```
import os
from langchain.chat_models import init_chat_model

os.environ["GROQ_API_KEY"] = os.getenv("GROQ_API_KEY")

model = init_chat_model("groq:openai/gpt-oss-20b")

model
```

```
EXECUTION OUTPUT (MODEL PROFILE)

ChatGroq(metadata={'lc_versions': {'langchain-core': '1.7.0', 'langchain': '1.4.7'}, profile={'name': 'GPT OSS 20B',
'release_date': '2025-08-05', 'open_weights': True, 'max_input_tokens': 131072, 'max_output_tokens': 65536, 'text_inputs':
True, 'image_inputs': False, 'tool_calling': True, 'structured_output': True})
```

Key Mechanics:

- The model profile confirms native support for `'tool_calling': True` and `'structured_output': True`, enabling reliable schema enforcement without fallback prompt hacks.
- `max_input_tokens: 131072` provides substantial headroom for ingesting large documents while extracting structured records.

Cell 2: Markdown Section Header

```
### Pyantic
```

Analysis: Focuses on Pydantic v2 schema definitions.

Cell 3: Flat Pydantic Model Definition & Structured Invocation

```
from pydantic import BaseModel, Field

class Movie(BaseModel):
    title: str = Field(description="title of the movie")
    year: int = Field(description="year the move was released")
    director: str = Field(description="director of the movie")
    rating: float = Field(description="movie rating out of 10")

model_with_structure = model.with_structured_output(Movie)

response = model_with_structure.invoke("Tell me about movie avengers endgame")

response
```

```
EXECUTION RESULT (VALIDATED OBJECT)

Movie(title='Avengers: Endgame', year=2019, director='Anthony Russo', rating=8.4)
```

Detailed Analysis:

- `Field(description="...")`: Crucial pedagogical element. LangChain exports these descriptions directly into the JSON Schema. The LLM reads these descriptions to determine what data belongs in each property.
- `response` is *not* a string or dictionary; it is a live `Movie` Pydantic class instance. Developers get complete IDE autocomplete: `response.title`, `response.year`, `response.rating`.

Cell 4: Dual Output Inspection via include_raw=True

```
# Pydantic can also include raw message along with structure
model_with_structure = model.with_structured_output(Movie, include_raw=True)

response = model_with_structure.invoke("Tell me about movie avengers endgame")

response
```

```
EXECUTION RESULT (DICTIONARY WITH RAW AND PARSED)

{'raw': AIMessage(content='', additional_kwargs={'reasoning_content': 'We need to output a response. The user wants
information about the movie "Avengers: Endgame". We can use the function. The function requires director, rating, title,
year. We\'ll call the function.', 'tool_calls': [{'id': 'fc_cfe...', 'function': {'arguments': '{"director":"Anthony
Russo","rating":8.4,"title":"Avengers: Endgame","year":2019}', 'name': 'Movie'}, 'type': 'function'}]}, response_metadata=
{'token_usage': {'completion_tokens': 98, 'prompt_tokens': 165, 'total_tokens': 263}}),
 'parsed': Movie(title='Avengers: Endgame', year=2019, director='Anthony Russo', rating=8.4),
 'parsing_error': None}
```

Architectural Rationale:

- Setting `include_raw=True` returns a 3-key dictionary: `raw`, `parsed`, and `parsing_error`.
- **Observability & Auditing:** The `raw` key preserves the underlying `AIMessage`, complete with token usage and internal reasoning. If validation fails, `parsed` is `None` and `parsing_error` captures the exact Pydantic `ValidationError` for graceful recovery.

Cell 5: Nested Pydantic Hierarchies & Optional Fields

```
# Nested pydantic structure
class actor(BaseModel):
    name: str
    role: str

class movie(BaseModel):
    title: str
    year: int
    cast: list[actor]
    genres: list[str]
    budget: float | None = Field(None, description="Budget In Millions USD")

model_with_structure = model.with_structured_output(movie)

response = model_with_structure.invoke("Tell me about movie avengers endgame")

response
```

```
EXECUTION RESULT (NESTED OBJECT)

movie(title='Avengers: Endgame',
      year=2019,
      cast=[actor(name='Robert Downey Jr.', role='Tony Stark / Iron Man'),
            actor(name='Chris Evans', role='Steve Rogers / Captain America'),
            actor(name='Mark Ruffalo', role='Bruce Banner / Hulk'),
            actor(name='Chris Hemsworth', role='Thor'),
            actor(name='Scarlett Johansson', role='Natasha Romanoff / Black Widow')],
      genres=['Action', 'Adventure', 'Drama', 'Sci-Fi'],
      budget=356.0)
```

Advanced Modeling Features:

- `cast: list[actor]`: The model automatically formats an array of JSON objects matching the `actor` schema.
- `budget: float | None = Field(None, ...)`: Declares an optional attribute with a default of `None`. If the model does not know the budget, it will not fail validation.

Cell 6: Markdown Section Header

```
### TypedDict
```

Analysis: Transitioning to lightweight dictionary-based typing.

Cell 7: Flat TypedDict Schema Definition

```
from typing_extensions import TypedDict, Annotated

class movie(TypedDict):
    """a movie with details"""
    title: Annotated[str, ..., "title of the movie"]
    year: Annotated[int, ..., "year the move was released"]
    director: Annotated[str, ..., "director of the movie"]
    rating: Annotated[float, ..., "movie rating out of 10"]

model_with_structure = model.with_structured_output(movie)

response = model_with_structure.invoke("Tell me about avengers endgame")

response
```

```
EXECUTION RESULT (PURE PYTHON DICTIONARY)

{'director': 'Anthony Russo and Joe Russo',
 'rating': 8.4,
 'title': 'Avengers: Endgame',
 'year': 2019}
```

Key Mechanics: `Annotated[type, ..., "description"]` injects field descriptions into the JSON Schema without requiring Pydantic. The result is a standard Python `dict`.

Cell 8: Nested TypedDict Schema

```
# Nested TypedDict structure
class actor(TypedDict):
    name: str
    role: str

class movie(TypedDict):
    title: str
    year: int
    cast: list[actor]
    genres: list[str]
    budget: float | None = Field(None, description="Budget In Millions USD")

model_with_structure = model.with_structured_output(movie)

response = model_with_structure.invoke("Tell me about movie avengers endgame")

response
```

```
EXECUTION RESULT (NESTED DICTIONARY)

{'budget': 356000000,
 'cast': [{'name': 'Robert Downey Jr.', 'role': 'Tony Stark / Iron Man'},
          {'name': 'Chris Evans', 'role': 'Steve Rogers / Captain America'},
          {'name': 'Mark Ruffalo', 'role': 'Bruce Banner / Hulk'},
          {'name': 'Chris Hemsworth', 'role': 'Thor'},
          {'name': 'Scarlett Johansson', 'role': 'Natasha Romanoff / Black Widow'}],
 'genres': ['Action', 'Adventure', 'Sci-Fi'],
 'title': 'Avengers: Endgame',
 'year': 2019}
```

Observation: Returns a clean nested dictionary structure. Notice that while Pydantic returned `356.0` (millions), here the model emitted `356000000` integer.

Cell 9: Markdown Section Header

```
### DataClasses
```

Analysis: Exploring Python dataclasses integrated into LangChain agents.

Cell 10: Agent Structured Output via response_format

```
from dataclasses import dataclass
from langchain.agents import create_agent

@dataclass
class ContactInfo(BaseModel):
    """Contact information for a person."""
    name: str = Field(description="The name of the person")
    email: str = Field(description="The email address of the person")
    phone: str = Field(description="The phone number of the person")

model = init_chat_model("groq:openai/gpt-oss-20b")

agent = create_agent(
    model=model,
    response_format=ContactInfo  # Auto-selects ProviderStrategy
)

result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567"
    }]
})

result
```

```
AGENT STATE OUTPUT (EXCERPTS)

{'messages': [HumanMessage(content='Extract contact info from: John Doe, john@example.com, (555) 123-4567'),
              AIMessage(content='{"name":"John Doe","email":"john@example.com","phone":"(555) 123-4567"}', ...)],
 'structured_response': ContactInfo(name='John Doe', email='john@example.com', phone='(555) 123-4567')}
```

Architectural Breakthrough: In modern LangChain agents, passing `response_format=ContactInfo` automatically instructs the agent's `CompiledStateGraph` to extract and validate the final answer into a strongly-typed schema.

Cell 11: Accessing the Validated Agent Response

```
result["structured_response"]
```

```
EXECUTION RESULT

ContactInfo(name='John Doe', email='john@example.com', phone='(555) 123-4567')
```

Explanation: The application extracts the typed entity directly via `result["structured_response"]` without needing to parse the raw text inside `result["messages"]`.

F. End-to-End Structured Output Pipeline Diagram

The diagram below traces the end-to-end lifecycle of schema compilation, model function-calling constraints, and validation:

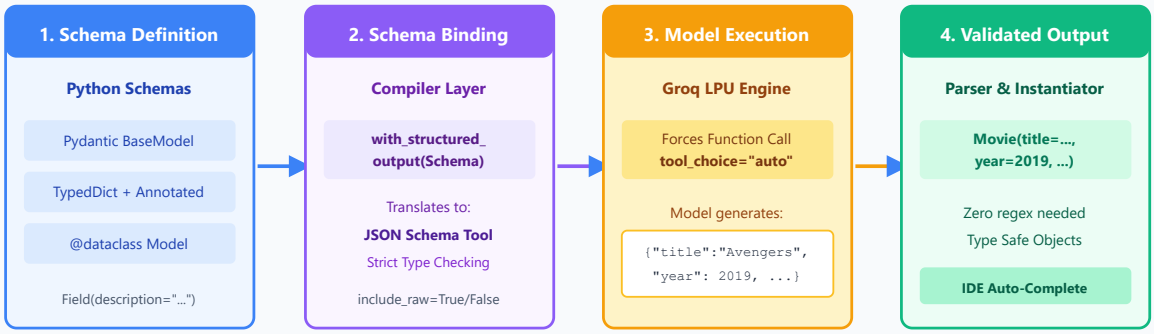


Figure 5.1: LangChain Structured Output Compilation and Validation Pipeline

G. Important APIs & Classes Reference

API / Function	Module / Class	Parameters & Flags	Description & Key Role
`with_structured_output`	`BaseChatModel`	`schema`, `include_raw=False`, `method="function_calling"`	Compiles a schema into a model-enforced output wrapper returning validated objects.
`BaseModel`	`pydantic`	Class inheritance	Primary schema base class providing type enforcement and automated JSON serialization.
`Field`	`pydantic`	`description`, `default`, `ge`, `le`	Attaches semantic field documentation passed into the LLM's function definition.
`TypedDict`	`typing_extensions`	Dictionary typing	Lightweight schema alternative returning native Python dictionaries.
`Annotated`	`typing_extensions`	`Annotated[type, ..., description]`	Injects documentation metadata into TypedDict fields for model schema generation.
`response_format`	`langchain.agents.create_agent`	`schema=Class`	Instructs an autonomous agent graph to produce a validated `structured_response`.

H. Common Errors, Debugging & Troubleshooting

Trap 1: Missing Field Descriptions Lead to Hallucinated Data

Symptom: The model places irrelevant data into fields or guesses incorrect formats.

Root Cause: If a model has `budget: float` without a description, the LLM does not know if the unit is Dollars, Euros, Millions, or Exact Cents. In Cell 5, `Field(description="Budget In Millions USD")` ensured `356.0` rather than ambiguous integer units.

Fix: Always provide explicit descriptions for every field in Pydantic or TypedDict.

Trap 2: Strict Type Validation Rejections

Symptom: `pydantic_core._pydantic_core.ValidationError: Input should be a valid integer`.

Root Cause: The model emits `"year": "2019 (May)"` as a string when an `int` was strictly required.

Fix: Use `include_raw=True` to inspect the failure, or use flexible Pydantic field validators / string sanitizers before coercion.

I. Practical Example: Enterprise Extractor with Validation Fallback

```
from pydantic import BaseModel, Field, ValidationError

class UserProfile(BaseModel):
    full_name: str = Field(description="Full personal name")
    age: int = Field(description="Age in years")
    interests: list[str] = Field(description="List of primary hobbies or interests")

extractor = model.with_structured_output(UserProfile, include_raw=True)

raw_text = "Hi, I am Sarah Jenkins, twenty-eight years old, and I love hiking and photography."

result = extractor.invoke(raw_text)

if result['parsed'] is not None:
    profile: UserProfile = result['parsed']
    print(f"Extracted: {profile.full_name}, Age: {profile.age}, Interests: {profile.interests}")
else:
    print(f"Validation Failed: {result['parsing_error']}")
    print(f"Raw Model Payload: {result['raw'].additional_kwargs}")
```

J. Key Takeaways & Conceptual Map

- **Structure Replaces Regex:** `with_structured_output` provides validated Python objects directly from generative models.
- **Descriptions Are Functional Prompts:** Field descriptions are not comments; they are actively compiled into the LLM's function schema.
- **include_raw is Essential for Auditing:** Always use `include_raw=True` in production to retain token usage, reasoning traces, and error context.
- **Agents Support response_format:** In modern LangChain 1.4+, agents output structured records alongside full multi-turn conversational traces.

K. Practice Exercises & Knowledge Checks

Question 1 (Conceptual): What is the core difference between Pydantic and TypedDict when used with with_structured_output?

> **Answer:** Pydantic validates inputs and returns a strongly-typed class instance with methods and attribute access (`obj.title`). `TypedDict` performs compile-time type-hinting and returns a standard Python dictionary (`obj['title']`).

Question 2 (Code Tracing): What does response['raw'] contain when include_raw=True is passed?

> **Answer:** It contains the underlying `AIMessage` instance emitted by the model, including `content` , `tool_calls` , `response_metadata` (token usage, execution time), and `additional_kwargs['reasoning_content']` .

Question 3 (Implementation): How do you define an optional field in Pydantic v2 with a default of None?

> **Answer:** `field_name: type | None = Field(None, description="...")` . Specifying `None` as the default value allows Pydantic to instantiate the model even if the LLM omits the key.

L. Further Learning & Official Documentation

- LangChain Structured Outputs Guide
- Pydantic v2 Official Documentation
- Previous study guide in sequence: **4-messages.pdf** (Core Message Architecture).